Demo script (5 minutes)

For the demo rehearsal and the mock viva (T47). Rehearse it twice with a timer. One of you drives the app, the other talks; swap for the second rehearsal.

Before you start

Check	How
The laptop is plugged in	On battery it throttles and generation takes several times longer
The models are there	Nothing to do: the app loads them from `runs/`, or from the committed copies in `checkpoints/`. It says so at the top if one is missing
The app is open	`python run.py app` from the repository folder, then the browser tab it prints. Do this before the examiner arrives: the first start takes about 20 seconds. The app is light with blue controls (`.streamlit/config.toml`), which reads well on a projector
One warm-up click	Press **Generate layouts** once, so the first real click answers in about a second
The backups are open	`reports/demo/` and `reports/figures/` in a file window, in case the app fails. `reports/demo/` holds a screenshot of the app at each step of this script (`app_layouts.png`, `app_refused.png`, `app_compare.png`, `app_pinned.png`, `app_results.png`) and three exported layouts

The script

0:00 The problem (30 seconds). Leave the app on its start page.

> "Most AI room tools draw a picture. A picture can look fine while the sofa blocks the door. We treat a layout as data: each item has a position and a facing, so a layout can be checked exactly. A CVAE proposes 64 layouts, gradient steps in its latent space repair them, a rule checker verifies every one, and a CNN ranks the valid ones."

0:30 A first room (60 seconds). Sidebar defaults: 5 × 4 m, door on the west wall, sofa, TV unit and coffee table. Press **Generate layouts**.

- Point at the caption: how many of the 64 candidates passed every check, and the time.
- Point at one layout: the two large numbers (its rule quality, and the score the CNN evaluator gave it), the cost and the floor covered below them, and the four checks, all "Pass".
- Open **Quality terms** on one layout. Say what the weakest term is.
- "Each layout can be downloaded as JSON or as a picture."

1:30 What it refuses (45 seconds).

- Set the width to 3.0 and the depth to 3.0, and tick every piece of furniture. Press **Generate layouts**. The app says the furniture covers too much of the floor and suggests removing an item. "It never shows a bad layout instead."
- Set the budget to 20,000 with the default room: it names the cheapest possible cost.
- Set the width to 7.8: the warning says the room is outside the training range. "It still works there; our experiment E10 measured how well."

2:15 Compare methods (75 seconds). Reset to the default room, open **Compare methods**, press the button.

- "Five methods, the same room, 64 samples each."
- B1, random placement: a handful valid (6 of 64 with the default room and seed). B2, copying positions from similar training rooms: more than half (50).
- M1, the CVAE alone: fewer than half (25). M2, after latent optimization: most (57). "That step is what makes the neural pipeline work. Over 500 test rooms it takes the valid share from 17% to 70%."
- G0, our rule-based generator: "It is the reference. It is valid by construction and about nine times cheaper per layout. We say that openly: for an ordinary request the generator is the better tool."

3:30 Pinning an item (45 seconds). Back on **Layouts**. In the sidebar choose **Keep this item where I put it: Sofa**, move it towards the north wall (second slider near 0.85), facing south. Press **Generate layouts**.

- "The sofa is exactly where I put it in all three, and the rest is arranged around it."
- Say the honest part: "We measured this on 360 requests. The network honours the pin and almost always finds a valid layout, but our rule-based generator with the item simply moved does it better: 47% of its samples are valid against 39%."
- Layouts 2 and 3 are clearly worse here (rule quality 0.42 and 0.55 against 0.89 for Layout 1). If asked why: the three layouts must differ by 0.3 m per item on average, and with the sofa fixed the best candidates are near-copies of Layout 1, so the next two picks are the 15th and the 18th of the 40 valid candidates in the CNN's order. The CNN also overrates them (0.78 for Layout 2, whose TV unit faces the wall). That is why the app shows the rule quality next to the evaluator's score.

4:15 Results (45 seconds). Open **Training and results**.

- Show *E1: every method on the same test rooms*. One sentence: M2 beats both samplers on validity; the generator stays ahead on quality and cost.
- Show *E8: latent-optimization steps*. "Most of the gain comes in the first 25 steps."
- If asked for more: *E3b* (vanishing gradients and dying ReLU without BatchNorm) and *Gate 2* (why we chose MAE, and why not Tanh).

5:00 Stop. "Everything shown regenerates from a fixed seed. We ran every step again into an empty folder: all 21 tables, all 64 trained models and the dataset came out identical. And the report's appendix derives every formula."

If something goes wrong

Problem	What to do
The app does not start	Go through the screenshots `reports/demo/app_layouts.png`, `app_refused.png`, `app_compare.png`, `app_pinned.png` and `app_results.png` in the order of this script; or run `python run.py generate --width 5 --depth 4 --door W --items sofa,tv_unit,coffee_table` in a terminal and show `reports/demo/top1.png` to `top3.png`
Generation is slow	The laptop is on battery or hot. Lower **Candidates** to 32
"No valid layout" for a room	That is honest behaviour: say so, then change the seed or remove an item
A question about a number	Open `reports/viva_prep.md`, Section 2.3

Questions to expect during the demo

- **"Is the layout guaranteed valid?"** Yes: only layouts that pass the rule checker are shown. The networks never decide validity.
- **"Why are the three layouts different?"** The top 3 must be at least 0.3 m apart on average per item; otherwise the three best would be near-copies.
- **"Why did M1 fail so often?"** A decoder trained to reconstruct positions is off by about 0.2 m per item, which is enough for a sofa to touch a table. Latent optimization removes exactly those small violations.
- **"What is the CNN doing here?"** Ranking the valid layouts. Ranked at random, the top 3 would score 0.67; with the CNN 0.85; with the exact rule score 0.88.
- **"Why do the two numbers under a layout differ?"** The rule quality is computed exactly from the layout. The evaluator's score is the CNN's estimate of it from a picture of the room, and it decides the order. They agree well on ordinary requests and less well when an item is pinned to an unusual spot, which the CNN never saw in training.